

CNN Accelerator - Dataflow

Rahul B.

August 7, 2026

1 Line Buffers and Sliding Window

- **How images are stored:**

Images are stored in memory in raster-scan order, which is a row by row storing method. When the image is read it happens on a single 1D stream, row by row.

- **What is needed:**

What the CNN accelerator is doing in a nutshell is, reading a 3x3 pixel block from an image in a single clock cycle so it can be convoluted with a weighting function to get the output in a single clock cycle. So there must be a way to turn this 1D pixel stream into a stream which outputs 3x3 pixels on every clock.

- **The Solution:**

The solution to this problem is using line buffers, which holds the pixels of a row by utilizing delays. For a 3x3 matrix, we need 2 line buffers. The first 2 rows of pixels will be filled with the 2 buffers while the third row of pixels will be coming in. The output is read from each line buffer and from the direct pixel stream itself. So we get a 3x3 pixel matrix on every clock cycle. Important thing to note here is the line buffers are connected in series and the outputs are taken out in parallel.

- **Sliding Window:**

The line buffer only solved the vertical storing, i.e it stores each pixel from each row. We need a solution for horizontal storing as well. We use an array of registers (3x3 registers) to solve the horizontal storing. Each row of registers are connected to each other, so after they are filled at each clock cycle 3x3 pixels will be sent to the next stage of the accelerator. These rows of registers are called the 3x3 window.

- **Implementations of the Line Buffer:**

The line buffer can be implemented in two ways, one way is to use an array of registers which is perfect for small size images (Width of 64 or less) to be used in testing. Second way is to take advantage of BRAM blocks on the FPGA to create the line buffer.

- **Some Edge Cases to be aware about:**

- **Pipeline Latency:**

At the very start when a pixel enters, the outputs of the line buffers will be invalid. So there should be a control logic incorporated to have a grace period or a control signal which will inform the next stage on the path to hold until the buffers get filled up.

- **Stalls:**

If the upcoming stages get stuck for some reason, the linebuffers and the windowing mechanism has to stop and wait. So an AXI stream handshake process needs to be implemented to avoid this problem.

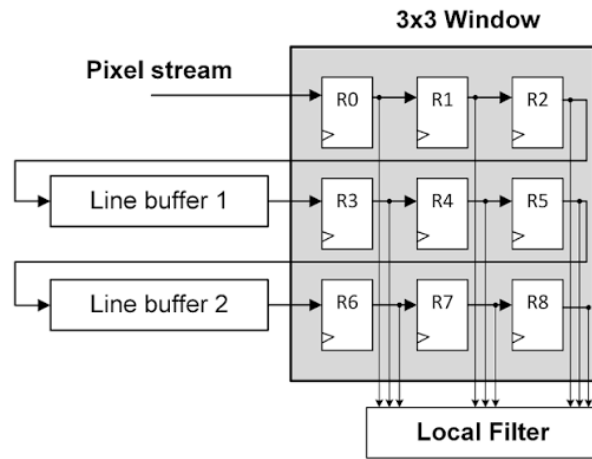


Figure 1: Line Buffer with a 3x3 window